CYBERSECURITY BOOTCAMP · CAPSTONE REPORT

Automated Web Security Scanner for Security Headers and CORS Misconfigurations

An Automated Vulnerability Assessment Tool for Defensive HTTP Header & Cross-Origin Configuration Analysis

---

Author / Candidate:	Zoya Tahareen
GitHub Username:	zoyatahareen
Project Repository:	https://github.com/ZoyaTahareen/Web-Security-Scanner
Submission Track:	Cybersecurity Engineering / Web Application Security
Application Framework:	Python 3.12 + Flask 3.0.3 (WSGI / Gunicorn / Render Ready)
Test Suite Verification:	**102 Passed, 0 Failures** (Pytest with 100% Mocked Network)
Report Date:	October 22, 2026
Deliverable Scope:	Working Web App, RESTful Scanner, Pure-Black GUI, 3 Exporters, 102 Unit Tests

Executive Summary: Modern web applications frequently expose users and data to attack due to improperly configured HTTP security response headers and permissive Cross-Origin Resource Sharing (CORS) policies. This project implements an automated, defensive security scanner capable of validating target endpoints with Server-Side Request Forgery (SSRF) protections, passively evaluating 5 critical security headers and 4 CORS directives, computing transparent rule-based risk severity rankings, and exporting audit reports in JSON, CSV, and PDF formats through a dedicated pure-black themed cybersecurity dashboard.

1. Abstract & Introduction

Abstract: HTTP security headers provide a primary browser-enforced defense against common client-side vulnerabilities including Cross-Site Scripting (XSS), clickjacking, MIME-type confusion attacks, and insecure protocol downgrades. In tandem, Cross-Origin Resource Sharing (CORS) governs browser authorization boundaries for cross-domain AJAX requests. Despite well-established industry standards (OWASP, W3C, RFCs), real-world web applications routinely omit defensive headers or implement dangerous CORS rules (such as wildcard reflections with credentials). This project presents an end-to-end, reproducible, defensive web security inspection platform developed in Python and Flask.

Background: Security teams and penetration testers conduct initial passive reconnaissance before executing active tests. Manually inspecting raw HTTP response headers using terminal commands (e.g., curl -I) or browser developer tools is tedious, error-prone, and fails to systematically categorize findings by risk level or generate client-ready reports. The Automated Web Security Scanner streamlines this workflow into an interactive, defensive, and accessible software platform.

2. Problem Statement & Project Objectives

The Problem: The majority of web vulnerabilities identified in bug bounty programs and security audits stem not from complex zero-day flaws, but from basic misconfigurations. Without strict Content Security Policy (CSP), attackers can execute malicious scripts; without HTTP Strict Transport Security (HSTS), traffic is vulnerable to SSL stripping; without proper CORS controls, malicious third-party origins can read authenticated responses. Existing open-source tools often require complex CLI setups, lack SSRF protection for hosted environments, or do not provide formatted, multi-type downloadable reports.

Primary Project Objectives:

- **Automate Security Header Inspection:** Systematically inspect 5 core security headers: CSP, HSTS, X-Content-Type-Options, X-Frame-Options, and Referrer-Policy.
- **Analyze Cross-Origin Resource Sharing (CORS):** Inspect Origin, Credentials, Methods, and Allowed Headers for wildcards, null origins, and overly permissive patterns.
- **Enforce Robust SSRF Protections:** Reject loopback (127.0.0.0/8, ::1), private IP blocks (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), link-local addresses (169.254.0.0/16), and cloud metadata endpoints (AWS, GCP, Azure).
- **Rule-Based Severity Classification:** Assign High, Medium, Low, or Informational severities using deterministic, documented rules rather than arbitrary scores.
- **Professional Interactive Dashboard:** Deliver a responsive, pure-black themed web user interface with dynamic tab switching and real-time status indication.
- **Multi-Format Report Export:** Allow auditors to export findings into machine-readable JSON, spreadsheet-ready CSV, and formal styled PDF reports.
- **Thorough Automated Testing:** Ensure reliability with an exhaustive Pytest suite (102 tests) verifying all validation rules, mock HTTP responses, and edge cases.

3. Architecture & Technology Stack

Component	Technology / Library	Architectural Role & Justification
Backend Framework	Flask 3.0.3 (Python)	Lightweight WSGI framework providing clean routing, session management, and JSON APIs.

HTTP Engine	Requests 2.32.3	Robust client with fine-grained timeout controls, TLS verification, and redirect inspection.
SSRF Defense	Python ipaddress + socket	Validates public FQDNs, resolves DNS, and blocks RFC 1918 / RFC 3927 / cloud metadata ranges.
Report Engine	ReportLab 4.2.2	Programmatic, vector-based PDF generation engine creating styled audit documents.
Testing Suite	Pytest 8.3.3	Automated unit and integration test framework with mock networking and fixture support.
Production WSGI	Gunicorn 22.0.0	Multi-worker production server configured for Render cloud platform deployment.
Frontend Interface	HTML5 / Vanilla CSS3 / JS	Single-page black aesthetic UI (#000000 base, #00ff88 accents) without bulky JS frameworks.

Workflow Pipeline:

1. **Client Input:** Target URL submitted via UI or JSON POST request to /scan.
2. **Validation & SSRF Filter:** Scheme verified (HTTP/HTTPS only), hostname extracted, DNS resolved; private/loopback/cloud metadata IPs rejected immediately with HTTP 400.
3. **Passive HTTP Probing:** Requests dispatched with explicit 10s timeouts, TLS certificate validation enforced, and redirect chain verified against SSRF rules.
4. **Header & CORS Analysis:** Inspection rules analyze presence, directives, and parameter values.
5. **Risk Scoring:** Findings assigned severity (High, Medium, Low, Info) with explicit evidence and remediation advice.
6. **Session Persistence & Reporting:** Complete audit structure stored in session; client downloads JSON, CSV, or PDF on demand.

4. Detailed Implementation Modules

4.1 URL Validator & SSRF Protection (scanner/validator.py)

Server-Side Request Forgery is a critical web risk when servers make outbound requests based on user input. The validator implements defense-in-depth before any socket connection is opened:

- **Strict Scheme Allowlist:** Only http:// and https:// schemes are allowed. Unsafe schemes like ftp://, file://, gopher://, and data: are rejected.
- **Format Sanitization:** Hostnames are extracted cleanly; trailing whitespace and URL fragments are stripped.
- **Prohibited Target Filtering:** Explicitly blocks localhost, 127.0.0.1, [::1], and cloud metadata hostnames (e.g., metadata.google.internal, 169.254.169.254).
- **DNS Resolution & CIDR Verification:** Resolves hostnames to IP addresses via socket.getaddrinfo() and passes each IP through Python's ipaddress.ip_address(). Any address that is private (RFC 1918: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), loopback (127.0.0.0/8), link-local (169.254.0.0/16), or multicast is blocked.

4.2 HTTP Security Headers Inspection Engine (scanner/headers.py)

The header analyzer inspects 5 fundamental security headers using context-aware heuristics:

Header Name	Expected Directive / Value	Security Implication & Status Logic
Content-Security-Policy	Restrictive source allowlist (default-src, script-src)	Mitigates XSS and data injection. WARNING if missing; REVIEW if unsafe-inline or unsafe-eval present.
Strict-Transport-Security	max-age >= 31536000; includeSubDomains	Prevents SSL stripping. WARNING if missing on HTTPS; REVIEW if max-age < 1 year; N/A on plain HTTP.
X-Content-Type-Options	nosniff	Prevents MIME-confusion attacks and drive-by downloads. WARNING if absent; PASS only if 'nosniff'.
X-Frame-Options	DENY or SAMEORIGIN	Guards against Clickjacking attacks. WARNING if missing (unless CSP frame-ancestors is present).
Referrer-Policy	strict-origin-when-cross-origin / no-referrer	Controls sensitive path/token leakage in Referer header. REVIEW if default/missing; WARNING if unsafe-url.

4.3 Cross-Origin Resource Sharing (CORS) Module (scanner/cors.py)

CORS controls allow browsers to permit cross-domain resource reads. Misconfigured headers can turn authenticated internal APIs into public data leaks. The module passively analyzes:

- **Access-Control-Allow-Origin:** Flags wildcard (*) configurations as REVIEW for public data. Flags null origin as WARNING due to sandboxed iframe bypass vectors.
- **Access-Control-Allow-Credentials:** Evaluates combination of true credentials with specific or wildcard origins. Emphasizes that browsers block * + credentials: true, but dynamic origin reflection requires immediate investigation.
- **Access-Control-Allow-Methods & Headers:** Evaluates unsafe HTTP verbs (e.g., PUT, DELETE) and broad header reflections (e.g., Authorization).

4.4 Risk Assessment & Multi-Report Exporter (scanner/risk_assessment.py, report_generator.py)

Each identified issue produces a structured finding dictionary containing: Title, Severity, Evidence, Technical Explanation, Business/Exploit Impact, and Actionable Remediation Guidance. Findings are categorized into four transparent tiers:

- **High Severity:** Missing core defenses on sensitive interfaces (e.g., missing CSP + frame protections, unsafe CORS credentials).
- **Medium Severity:** Missing HSTS on HTTPS targets, absent X-Content-Type-Options, or overly permissive Referrer policies.
- **Low Severity:** Weak parameter configurations (e.g., HSTS max-age under 1 year, legacy X-Frame-Options syntax).
- **Informational:** Best practice observations, verified secure configurations, and baseline architectural notes.

5. Interactive Web Dashboard & Pure-Black Aesthetic

In accordance with modern offensive/defensive security tools, the user interface was engineered with a dedicated **pure-black cybersecurity theme**. The aesthetic emphasizes clarity, high contrast, and reduced visual fatigue during auditing sessions.

Interface Layer	Design Implementation & Color Specification
Base Background	Pure absolute black (#000000) across body, panels, and modal containers.
Surface Layers	Deep charcoal elevations (#0a0a0a, #111111, #161616) with subtle border definition (#222222).
Primary Accent	Electric Cyber Green (#00ff88) for branding, active states, buttons, and pass badges.
Severity Palettes	High: #ff4444 | Medium: #ffaa00 | Low: #00c8ff | Info: #888888.
Interaction Features	Tab navigation (Headers, CORS, Findings, Reports), live scan progress animation, copy-to-clipboard, responsive grid.

6. Testing Methodology & Actual Verification Results

A cornerstone of the project is its comprehensive automated test suite implemented in **Pytest**. To guarantee deterministic execution, zero network flakiness, and air-gapped testability, **100% of network interactions are mocked** using Python's unittest.mock.patch and mock HTTP response fixtures.

Test Module	Test Scope & Target Functionality	Tests Run	Status
tests/test_validator.py	URL schemes, DNS resolution, SSRF blocking (127.0.0.1, 10.0.0.0/8, 192.168.0.0/16, AWS metadata, IPv6 ::1).	26 tests	**PASSED**
tests/test_headers.py	CSP source checks, HSTS HTTPS context, X-Content-Type nosniff, X-Frame-Options deny/sameorigin, Referrer-Policy.	32 tests	**PASSED**
tests/test_cors.py	Wildcard origins, null origins, credential pairing, dangerous HTTP verbs, authorization header reflections.	24 tests	**PASSED**
tests/test_app.py	Flask endpoints (/, /scan), input validation, session persistence, JSON/CSV/PDF download handlers.	20 tests	**PASSED**
TOTALS	**Complete Project Test Suite**	**102 tests**	**102 / 102 PASSED (100%)**

Actual Pytest Execution Snapshot:

```
collected 102 items
tests/test_cors.py ..................... [ 23%]
tests/test_headers.py ............................. [ 55%]
tests/test_validator.py ....................... [ 80%]
tests/test_app.py ................... [100%]
======================= 102 passed, 2 warnings in 1.10s =======================
```

7. Sample Security Findings & Remediation Walkthrough

Finding: Missing Content-Security-Policy (CSP) **Severity: HIGH**

Evidence: Header absent from response

Risk Explanation: The target does not deliver a CSP header. Browsers will execute arbitrary inline scripts and load resources from unverified origins if an XSS injection occurs.

Remediation: Add header: Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none';

Finding: Strict-Transport-Security (HSTS) Missing on HTTPS **Severity: MEDIUM**

Evidence: Observed scheme: https://, header absent

Risk Explanation: Users communicating over HTTPS are vulnerable to man-in-the-middle downgrade attacks (SSL stripping) on their initial unencrypted HTTP connection.

Remediation: Add header: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

Finding: X-Content-Type-Options Header Absent **Severity: MEDIUM**

Evidence: Header absent from response

Risk Explanation: User agents may perform MIME-type sniffing, treating non-executable files (e.g., images) as executable JavaScript if file content contains script tags.

Remediation: Add header: X-Content-Type-Options: nosniff

Finding: Permissive Wildcard Access-Control-Allow-Origin **Severity: REVIEW / INFO**

Evidence: Access-Control-Allow-Origin: *

Risk Explanation: The endpoint permits any external website to issue cross-origin requests and read responses. If this endpoint serves sensitive user data, this is a vulnerability.

Remediation: If the endpoint is an authenticated API, restrict the origin to specific trusted partner domains.

8. Project Scope, Limitations & Responsible-Use Policy

Assessment Limitations:

• *Passive Surface Analysis:* The scanner inspects publicly visible HTTP response headers. It does not perform active payload injection, directory fuzzing, or credential brute-forcing.

• *Context Sensitivity:* A missing header on a purely static landing page presents lower practical risk than on a multi-tenant banking application. Context must be evaluated by a certified human analyst.

• *Single-Point Snapshot:* Scans represent server configuration at the exact moment of execution; CDN routing, load balancers, or geo-DNS may yield differing responses.

Responsible Use Disclaimer: This software is developed strictly for educational, defensive, and authorized assessment purposes. Scanning target systems without explicit authorization from the asset owner may violate federal, state, and international cyber legislation (e.g., Computer Fraud and Abuse Act 18 U.S.C. § 1030). Always obtain written permission prior to scanning third-party infrastructure.

9. Conclusion & Project Sign-Off

The Automated Web Security Scanner fulfills all requirements for the Cybersecurity Bootcamp Capstone Project. By pairing defensive SSRF validation with heuristic header and CORS analysis, transparent risk categorization, a bespoke pure-black interface, multi-format export engines, and an exhaustive 102-test automated verification suite, the project demonstrates practical mastery of web security, defensive architecture, and modern Python engineering.

Submitted By: Zoya Tahareen

Status: Completed & Verified

GitHub:
github.com/ZoyaTahareen/Web-Security-Scanner

Verification: 102 / 102 Pytest Suite Passed